INT4418 – Dữ liệu lớn · Bài tập lớn · Nhóm 7 (AI4Math)

Proof-Search Analytics: phân tích trace tìm kiếm chứng minh Lean 4, dự đoán tactic và cắt nhánh

Thành viên: [Họ tên 1 – MSSV] · [Họ tên 2 – MSSV] · [Họ tên 3 – MSSV] · [Họ tên 4 – MSSV]

Repo: proof-search-analytics/ · Mọi bảng/hình sinh từ results/raw bằng script (src/eval/figures.py, scripts/build_report.py).

Tóm tắt

Nhóm xây dựng một pipeline dữ liệu lớn cho trace tìm kiếm chứng minh: một prover best-first cố định chạy trên Lean 4 + mathlib, mỗi lần thử tactic thành một event theo schema có version, lưu vào trace lake Parquet, tổng hợp bằng PySpark, học predictor tactic và đánh giá pruning bằng replay rồi kiểm chứng live. Predictor TF-IDF + SGD đạt top-1 0.301 so với 0.203 của baseline most-frequent (top-10 0.851 vs 0.724). Trên 100 bài miniF2F ở budget M, pruning với $\tau = 0.01$ giảm states expanded từ 46.5 xuống 31.2 mỗi bài (−33%) mà proof success rate giữ nguyên 0.32 khi chạy Lean thật; replay cho thấy từ $\tau = 0.02$ proof rate bắt đầu giảm (0.25).

1. Bài toán và câu hỏi nghiên cứu

Tìm chứng minh tự động trong Lean là một bài toán tìm kiếm: tại mỗi proof state, prover thử nhiều tactic, phần lớn thất bại và mỗi lần thử tốn một lời gọi Lean. Dữ liệu sinh ra (state, tactic, kết quả, thời gian) lớn nhanh theo số định lý × số nhánh, và là nguồn để hiểu hành vi tìm kiếm cũng như học cách cắt nhánh. Đề tài tập trung vào phần dữ liệu: thu thập, lưu trữ, tổng hợp và đánh giá có kiểm soát chi phí.

- **RQ1.** Đặc trưng proof state dự đoán tactic tiếp theo tốt đến đâu (top-k), khác nhau thế nào theo domain?
- **RQ2.** Pruning bằng predictor giảm được bao nhiêu states expanded trước khi proof success rate giảm?

Primary metrics: proof success rate (chất lượng) và states expanded / định lý (chi phí). Chỉ số phụ: top-k accuracy, wall time, thời gian Lean, dung lượng lake, runtime tổng hợp, events/s.

2. Dữ liệu

Nguồn	Vai trò	Quy mô	Phiên bản / license
LeanDojo Benchmark 4 (Zenodo 10114185)	Analytics (E1), huấn luyện predictor (E2)	98,514 / 2,000 / 2,000 định lý (train/val/test, random split); 213,067 bước tactic	mathlib4 @ 3ce43c18; Apache-2.0
miniF2F Lean 4 (DeepSeek-Prover-V1.5)	Search thật (E3, E4)	488 bài; chọn cố định 100 bài (seed 42)	commit 2c4ba91; MIT
Trace tự sinh	Lake source=search	31,100 event (7 run hợp lệ)	Lean v4.9.0 + mathlib (LeanInteract 0.11.6)

Bảng 1. Nguồn dữ liệu. Dữ liệu đầy đủ không commit; repo có data_sample/ + manifest sha256 và script tải lại (make download, make minif2f).

Split cố định và có version: Benchmark dùng split của LeanDojo (chia theo định lý, tránh rò rỉ giữa các bước của cùng một chứng minh); tập 100 bài miniF2F được chọn bằng seed 42 sau khi

sắp xếp theo tên, lưu ở data/selections/. Mỗi run ghi data_version (vd minif2f-ds15@2c4ba91+lean-v4.9.0).

3. Kiến trúc hệ thống

#	Khối	Hiện thực	Điểm Big Data
1	Trace collector	src/search/{core,live_repl,collect}.py	best-first, budget 3 chiều (lời gọi, giây Lean, độ sâu); 2 worker song song; ghi staging rồi compact
2	Event schema	src/common/schema.py, docs/data_card.md	v0.1.0, 24 cột, grain = 1 lần thử tactic; validate() trước khi ghi
3	Parquet trace lake	src/ingest/benchmark.py, src/lake/manifest.py	partition source/budget_id, zstd, manifest sha256; 694 MB JSON → ~18 MB Parquet
4	Spark aggregation	src/analytics/aggregate.py	5 bảng; partition + column pruning, broadcast join, self-join, window lag
5	Feature extraction	src/model/train.py	TF-IDF n-gram trên proof state (≤200K đặc trưng)
6	Predictor / policy	src/model/{train,policy}.py	most-frequent vs TF-IDF + SGD; policy = P(tactic | state), cắt nếu < τ
7	Replay / evaluation	src/search/replay.py, src/eval/{prune_sweep,figures}.py	replay trace no-pruning cùng budget, ×3 lần; live check; Pareto

Bảng 2. Bảy khối của pipeline và nơi hiện thực trong repo.

Luồng dữ liệu: Benchmark JSON và trace search cùng đi vào một lake Parquet duy nhất (cùng schema), Spark đọc lake để sinh bảng tổng hợp CSV vào results/raw; predictor học từ phần benchmark; replay đọc trace no-pruning để mô phỏng pruning mà không cần Lean; figures.py sinh mọi hình/bảng kèm .meta.json (máy, phần mềm, seed, budget, số lần lặp). Mọi bước chạy trong Docker; mỗi run ghi một bản ghi Phụ lục B (run_id, git_commit, data_version, machine, software, parameters, seed, timeout, metrics, status) vào results/raw/runs.jsonl.

4. Schema và trace lake

Một dòng = một lần thử một tactic trên một proof state. Khóa (run_id, theorem_id, step_idx); parent_step_idx trỏ tới event sinh ra state_before nên dựng lại được cây tìm kiếm. Mỗi event mang budget của trace (budget_id, max_calls, timeout_s, max_depth) và policy/τ, nên không thể so sánh nhầm hai run khác budget. Dữ liệu append-only: sửa = tạo run mới.

Nhóm cột	Cột
Định danh, version	schema_version, source, run_id, data_version, split, theorem_id, file_path, domain
Cây tìm kiếm	step_idx, parent_step_idx, depth, state_before, state_after
Hành động & kết quả	tactic, tactic_rank, score, outcome (finished/progress/duplicate/error/unknown), elapsed_ms
Budget & policy	budget_id, max_calls, timeout_s, max_depth, policy, tau

Bảng 3. Schema v0.1.0 (chi tiết kiểu và quy tắc tiến hóa trong docs/data_card.md).

Quy tắc tiến hóa: thêm cột nullable ở cuối → tăng minor; đổi nghĩa/kiểu/xóa → tăng major. Nhóm khác đọc trực tiếp bằng pandas/pyarrow/Spark với filter theo partition; tính toàn vẹn kiểm bằng manifest sha256 (make sample-verify). Ingest toàn bộ Benchmark mất ~20 giây, ghi theo lô 5,000 định lý thành file lớn (tránh small-files problem); collect ghi file nhỏ vào staging rồi compact một lần cuối run.

5. Thiết lập thí nghiệm

Budget	Lời gọi tactic tối đa	Giây Lean tối đa / bài	Độ sâu tối đa
S	50	30	8
M	200	120	8
L	800	300	8

Bảng 4. Ba mức search budget (giống nhau cho baseline và pruning).

Máy: 8 nhân CPU, 19.5 GB RAM cấp cho Docker/WSL2, không GPU. Phần mềm: Python 3.11.17, Spark 3.5.3, scikit-learn 1.5.2, pyarrow 17.0.0, Lean 4.9.0 + mathlib qua LeanInteract. Danh sách ứng viên cố định (14 tactic): rfl, simp, norm_num, omega, ring, linarith, nlinarith, decide, positivity, simp_all, aesop, intro h, constructor, exact?. Mỗi lời gọi có timeout cứng 20 giây và maxHeartbeats 200,000. Seed 42 cho chọn bài, chia dữ liệu và huấn luyện.

	Biến thay đổi	Cố định	Đo
E1	100 → 1K → 10K → toàn bộ định lý; lake nhân bản ×10, ×50	Spark local[8], driver 6 GB	runtime median [min–max] ×3, events/s, dung lượng
E2	most-frequent vs TF-IDF+SGD; ablation unigram	split Benchmark, seed	top-1/3/5/10, theo domain
E3	budget S / M / L	100 bài, ứng viên, máy	proof rate, states, wall time
E4	$\tau \in \{0, 0.005, 0.01, 0.02, 0.05, 0.1, 0.2\}$	100 bài, budget M	Pareto proof rate ↔ states; bài bị mất

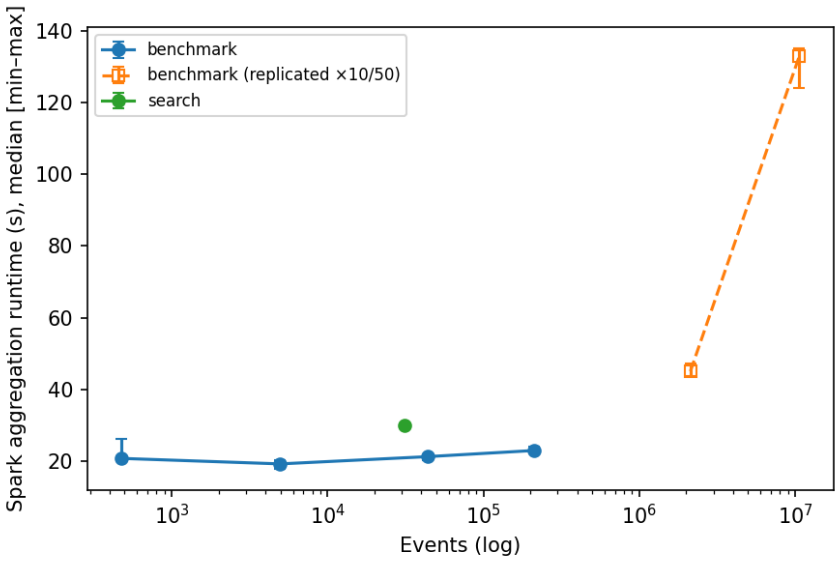
Bảng 5. Thiết kế thí nghiệm.

6. Kết quả

6.1 E1 – Khả năng mở rộng của Spark aggregation

Mức	Định lý	Event	Runtime s, median [min–max]	Event/s	Lake MB	Lần
100 định lý	100	476	20.8 [20.2–26.1]	23	18	3
1,000 định lý	1,000	4,978	19.2 [17.9–20.2]	259	18	3
10,000 định lý	10,000	43,815	21.2 [20.3–21.7]	2,062	18	3
toàn bộ	50,020	213,067	23.0 [22.9–23.9]	9,277	18	3
×10 (nhân bản)	500,200	2,130,670	45.1 [43.6–47.2]	47,281	173	3
×50 (nhân bản)	2,501,000	10,653,350	133.0 [124.2–135.2]	80,103	865	3

Bảng 6. Runtime tổng hợp 5 bảng (đọc + 5 action) theo quy mô.

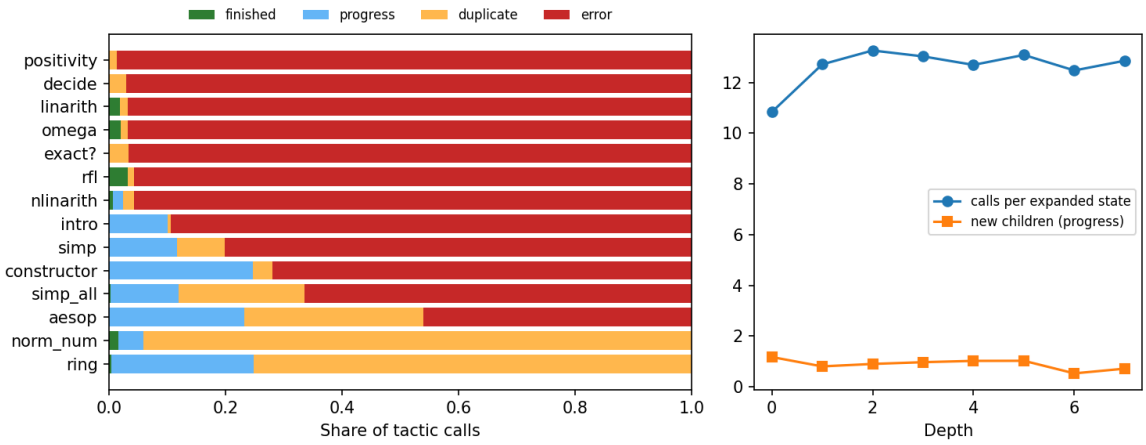


Hình 1. Runtime Spark theo số event (log). Đường nét đứt: lake nhân bản.

Với dữ liệu thật, runtime gần như phẳng: 21 s ở 476 event và 23 s ở 213,067 event. Ở quy mô này chi phí cố định (khởi động JVM, lập lịch stage, 5 lần collect về driver) lấn át. Để thấy vùng tuyến tính, nhóm nhân bản lake (mỗi bản sao đổi theorem_id thành <id>#k, nên groupBy/join vẫn tách đúng từng định lý). Ở ×50 (10,653,350 event, 865 MB) runtime là 133 s, throughput 80,103 event/s so với 9,277 event/s ở dữ liệu gốc. Dữ liệu nhân bản chỉ dùng đo runtime, không dùng cho bất kỳ kết quả chất lượng nào.

Physical plan (results/raw/spark_plans/, sinh bằng --explain): đọc lake chỉ lấy partition source=benchmark và 13 cột cần thiết (column pruning – không đọc text state_before/state_after). Lọc theo tập định lý là broadcast join (bảng keep nhỏ). Bảng transitions là self-join cha → con theo (run_id, theorem_id, step_idx): benchmark_n100: BroadcastHashJoin Inner, BroadcastHashJoin LeftOuter; benchmark_nall: SortMergeJoin LeftOuter. Khi phía cha nhỏ, AQE/optimizer chọn broadcast; khi cả hai phía lớn, Spark cần shuffle (Exchange hashpartitioning) trước khi join. Bảng position dùng Window lag theo (run_id, theorem_id) sắp theo step_idx, cũng cần shuffle theo khóa phân vùng.

6.2 Hành vi tìm kiếm (Spark trên trace search)



Hình 2. Trái: phân bố kết quả của 31,100 lời gọi tactic trong mọi run search hợp lệ. Phải: số lời gọi và số con mới (progress) trên mỗi state theo độ sâu.

Spark tổng hợp trace search (cùng code aggregate.py với phần Benchmark) cho thấy ba điều. (1) Phần lớn lời gọi thất bại: tỉ lệ error trung bình 74%. (2) norm_num trả về state đã gặp ở 94% lần gọi, ring trả về state đã gặp ở 75% lần gọi – chúng gần như không bao giờ báo lỗi mà chuẩn hóa goal về đúng dạng cũ; riêng các lời gọi này chiếm 12% tổng lời gọi, là chi phí Lean vô ích mà một bộ lọc 'no-op' rẻ hơn predictor có thể bỏ. (3) Tactic đóng bài nhiều nhất theo tỉ lệ: rfl (3.3%), omega (2.0%), linarith (1.9%). Chi phí mỗi lời gọi chênh nhau hơn 1229 lần: nhanh nhất decide (5 ms), chậm nhất exact? (5.5 s) – vì vậy budget đo bằng giây Lean chứ không chỉ bằng số lời gọi. Ở mọi độ sâu, mỗi state được thử ~13 tactic nhưng chỉ sinh ~0.9 con mới: cây hẹp, chi phí chủ yếu là các nhánh chết ngay.

6.3 E2 – Dự đoán tactic

Mô hình	Top-1	Top-3	Top-5	Top-10
Most-frequent (baseline)	0.203	0.453	0.567	0.724
TF-IDF 1–2 gram + SGD	**0.301**	**0.570**	**0.699**	**0.851**
Ablation: unigram	0.285	0.539	0.681	0.845

Bảng 7. Top-k accuracy trên test split Benchmark 4 (4,221 bước; train 204,205 bước).

Domain	Số bước test	Top-1 most-frequent	Top-1 TF-IDF	Chênh
Data	649	0.242	0.324	+0.082
MeasureTheory	460	0.159	0.307	+0.148
Analysis	395	0.165	0.263	+0.099
Algebra	344	0.285	0.337	+0.052
RingTheory	319	0.238	0.292	+0.053
Topology	286	0.157	0.283	+0.126
LinearAlgebra	260	0.188	0.254	+0.065
NumberTheory	223	0.215	0.283	+0.067

Bảng 8. Top-1 theo domain (8 domain nhiều bước nhất).

Predictor cải thiện top-1 thêm 0.098 tuyệt đối (+48% tương đối). Mức cải thiện không đều: lớn nhất ở MeasureTheory (+0.148), nhỏ nhất ở Algebra (+0.052). Bỏ bigram chỉ giảm top-1 0.015, tức phần lớn tín hiệu nằm ở từng token (ký hiệu, tên hằng) của goal. Chi phí: huấn luyện 242 s so với 0.6 s của baseline.

6.4 E3 – Search theo budget (no-pruning)

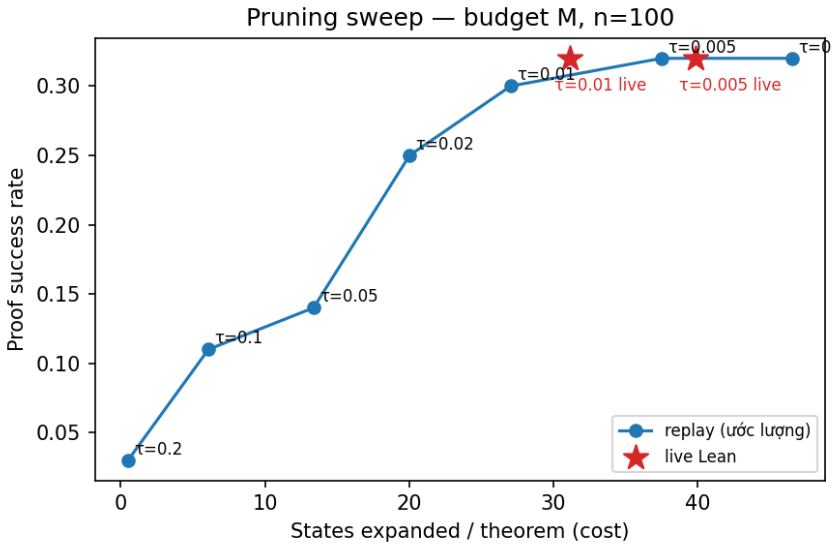
Run	Lần	Proof rate	States / bài	Wall s / bài
Baseline S	1	0.32	25.7	16.1
Baseline M	3	0.32	46.5 [44.5–46.5]	26.7 [22.3–36.6]
Baseline L	1	0.32	76.7	32.8
Pruning τ=0.005 (live, M)	1	0.32	39.8	20.5
Pruning τ=0.01 (live, M)	**1**	**0.32**	**31.2**	**18.9**

Bảng 9. Search thật trên 100 bài miniF2F. Nhiều lần chạy: median [min–max].

Tăng budget từ S lên L làm chi phí tăng 3.0× nhưng không chứng minh thêm bài nào. Tỉ lệ hết budget giảm từ 27% (S) xuống 4% (L): budget lớn chỉ giúp search duyệt hết cây sinh bởi 14

tactic cố định. Các bài còn lại nhiều khả năng cần tactic có tham số hoặc ngoài danh sách (vd interval_cases, hoặc nlinarith kèm bất đẳng thức gợi ý). Như vậy giới hạn chất lượng nằm ở không gian hành động, còn chi phí nằm ở số nhánh vô ích – đúng chỗ pruning có thể giúp.

6.5 E4 – Pruning: replay và kiểm chứng live



Hình 3. Pareto proof rate ↔ states expanded ở budget M. Đường xanh: replay (×3 lần, tất định); sao đỏ: chạy Lean thật với policy.

τ	Proof rate (replay)	States / bài	Unknown rate	Giây Lean ước tính / bài
0	0.32	46.5	0.00	21.5
0.005	0.32	37.5	0.05	19.0
0.01	0.30	27.0	0.07	16.3
0.02	0.25	20.0	0.10	13.7
0.05	0.14	13.4	0.20	10.7
0.1	0.11	6.1	0.15	5.0
0.2	0.03	0.5	0.02	0.3

Bảng 10. Replay sweep trên trace baseline M (cùng budget, cùng 100 bài).

Replay coi mọi lời gọi chưa có trong trace gốc là thất bại (unknown), nên là ước lượng bảo thủ. Ở τ = 0.01 replay dự đoán proof rate 0.30; chạy live đạt 0.32 với 31.2 states/bài (replay: 27.0). Replay mất 2 bài (mathd_algebra_104, mathd_algebra_412) vì policy cắt tactic của đường chứng minh gốc (linarith). Khi chạy live, cả hai được chứng minh bằng nlinarith – ứng viên đứng ngay sau trong danh sách mà baseline chưa từng thử vì đã thành công trước đó, nên không có trong trace. Đây chính là cơ chế làm replay bảo thủ.

Điểm gối nằm giữa τ = 0.01 và 0.02. Ở τ = 0.02 replay mất 7 bài so với baseline, cả 7 bài đều do policy cắt đúng tactic nằm trên đường chứng minh gốc; ở τ = 0.05 mất 18 bài, tactic bị cắt: rfl (10), linarith (6), norm_num (2). Các bài mathd_numbertheory (tính toán số cụ thể) bị ảnh hưởng nhiều nhất vì chứng minh gốc chỉ là rfl. Nguyên nhân: predictor học từ chứng minh của người trên mathlib, nơi rw, simp, exact chiếm 44% số bước, còn rfl chỉ 1.6%, linarith 0.3%, norm_num 0.3%. Prior thấp đó kéo xác suất của các tactic tự động xuống dưới ngưỡng, dù chúng giải trực tiếp nhiều bài thi đấu của miniF2F (lệch phân phối giữa nguồn huấn luyện và nơi áp dụng). Hướng cải tiến: fine-tune predictor trên trace search của chính prover.

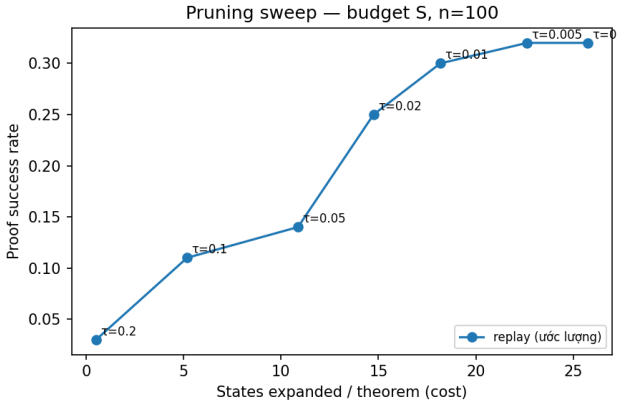
τ	proof tactic pruned
0.01	2
0.02	7
0.05	18
0.1	21
0.2	29

Bảng 11. Số bài bị mất theo nguyên nhân (src/eval/figures.py: table_pruning_losses).

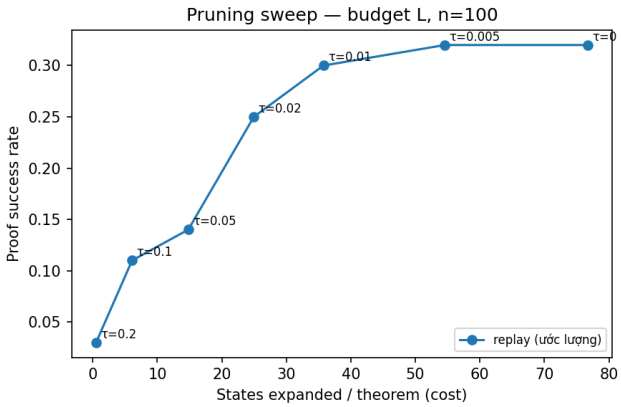
τ	S: proof / states	M: proof / states	L: proof / states
0	0.32 / 25.7	0.32 / 46.5	0.32 / 76.7
0.005	0.32 / 22.6	0.32 / 37.5	0.32 / 54.6
0.01	0.30 / 18.2	0.30 / 27.0	0.30 / 35.8
0.02	0.25 / 14.8	0.25 / 20.0	0.25 / 25.0
0.05	0.14 / 10.9	0.14 / 13.4	0.14 / 14.8

Bảng 12. Replay sweep ở ba mức budget (cùng 100 bài, cùng policy).

Theo budget, proof rate của replay ở mỗi τ giống hệt nhau giữa S, M và L (vd τ = 0.01: 0.30, 0.30, 0.30), vì cả 5 lần chạy baseline (S, M×3, L) chứng minh đúng cùng 32 bài, tức mọi bài giải được đều đã xong trong budget S. Ngược lại, phần states tiết kiệm tăng theo budget: S 29%, M 42%, L 53% ở τ = 0.01. Budget càng lớn, search càng tốn lời gọi cho các bài không giải được, và đó là phần policy cắt được. Hình 4 cho thấy hình dạng Pareto ở S và L giống M: điểm gối vẫn nằm giữa τ = 0.01 và 0.02, nên một ngưỡng chọn ở M dùng được cho cả ba mức budget.



Hình 4a. Pareto replay ở budget S.



Hình 4b. Pareto replay ở budget L.

Kết luận E4: nhóm chọn $\tau = 0.01$. So với baseline M cùng budget, cùng 100 bài, live giảm states expanded 33% và giây Lean 15%, không mất bài nào. Chi phí thêm của policy (một lần predict_proba mỗi state, có cache) không đáng kể so với lời gọi Lean. Không khẳng định 'nhanh hơn' cho $\tau \geq 0.02$ vì proof rate giảm.

7. Thảo luận và giới hạn

- **Quy mô search:** live search chỉ 100 bài (Lean chậm, ~20 s/bài ở budget M); quy mô 1K–10K+ nằm ở phần analytics trên Benchmark và lake nhân bản.
- **Replay là ước lượng:** lời gọi ngoài trace → unknown (tính thất bại). Vì vậy τ cuối cùng được kiểm chứng bằng chạy live, cùng budget và cùng bài.
- **Lệch phân phối:** predictor học từ chứng minh của người (mathlib 2023) nhưng chấm điểm ứng viên cố định trên state miniF2F (mathlib theo Lean v4.9.0).
- **Spark trên một máy:** local[8], không có cluster; kết quả scale phản ánh song song hóa trong một node. Dữ liệu nhân bản không có đa dạng thật, chỉ dùng đo runtime.
- **Độ ổn định:** proof rate và states gần như tất định; wall time phụ thuộc độ trễ Lean, nên baseline M được chạy 3 lần và báo median [min–max]. Search chạy tuần tự, không song song với job khác.
- **Sự cố đã xử lý:** phiên bản đầu của adapter REPL ghi sai mọi lời gọi sau một timeout cứng thành lỗi 0 ms. Lỗi được sửa (khởi động lại REPL và dựng lại state theo đường tactic), có test, và 5 run bị ảnh hưởng được cách ly kèm lý do trong results/raw/invalid_runs.json rồi chạy lại.
- **Tài nguyên máy:** hai lần chạy lặp baseline M bị ngắt giữa chừng (máy vào sleep; Windows hết bộ nhớ ảo vì không có pagefile, trong khi 2 REPL đã import Mathlib dùng ~7.5 GB). Collect chỉ ghi vào lake và runs.jsonl khi chạy xong, nên run dở không lọt vào dữ liệu; các lần lặp được chạy lại trọn vẹn.

Ghi chú lệch so với đề (deviation note).

Đề gợi ý thu trace bằng LeanDojo. LeanDojo phải trace toàn bộ mathlib4 (nhiều giờ, RAM và đĩa lớn), rủi ro cao trên máy cá nhân, nên phần search dùng LeanInteract (Lean REPL + mathlib build sẵn) trên miniF2F. Correctness: vẫn là Lean 4 + mathlib thật, cùng thuật toán và schema. Cost: không cần trace mathlib. Scalability: search ở 100 bài; quy mô lớn ở phần analytics. Integration: event cùng schema v0.1.0, source=search, data_version ghi rõ nguồn và phiên bản Lean; adapter LeanDojo vẫn có (make baseline).

8. Tái lập kết quả

Trong Docker (docker compose build), từ thư mục proof-search-analytics/:

```
make download sample ingest aggregate train ablation          # E1, E2
make replicate aggregate-rep                                  # E1 nhân bản
make minif2f select-minif2f search-all                        # E3, E4 (tuần tự, vài giờ)
make aggregate-search figures && python scripts/build_report.py
make dashboard                                                # http://localhost:8501
```

Mỗi hình có file .meta.json đi kèm (máy, phần mềm, seed, budget, số lần lặp, run nguồn). Kiểm thử: 27 test (pytest), gồm test replay τ=0 khớp trace gốc và test khôi phục REPL sau timeout.

9. Khai báo sử dụng AI và phân công

Nhóm dùng Claude Code (Anthropic) để sinh bản đầu của code, script thí nghiệm và tài liệu; chi tiết trong docs/ai_use.md. Mọi con số trong báo cáo được sinh từ results/raw bằng script, không lấy từ câu trả lời của AI. [Nhóm điền: ai đã đọc/kiểm chứng phần nào.] Phân công: xem contributions.md.